

Software Engineering

BCA — Semester 4 — 2020 — Answer Sheet

Subject Code: CACS253

Group B

Attempt any SIX questions. [6×5=30]

1. What is software engineering? Explain the challenges facing software engineering.

[2+3]

Answer: Software Engineering: Software engineering is the systematic application of engineering approaches to the development, operation, and maintenance of software. It encompasses techniques, methods, and tools for building high-quality software within budget and schedule constraints. Fritz Bauer defined it as "the establishment and use of sound engineering principles to obtain economically software that is reliable and works efficiently on real machines." **Challenges Facing Software Engineering:** 1. **Increasing Complexity:** Modern software systems are highly complex with millions of lines of code, multiple platforms, and distributed architectures. 2. **Changing Requirements:** Requirements evolve throughout the development lifecycle, making it difficult to deliver what customers actually need. 3. **Time and Cost Constraints:** Pressure to deliver quickly while maintaining quality and staying within budget. 4. **Quality Assurance:** Ensuring reliability, security, performance, and usability in diverse environments. 5. **Legacy Systems:** Integrating with and maintaining old systems while modernizing technology stacks. 6. **Security Threats:** Protecting against increasingly sophisticated cyber attacks and data breaches. 7. **Scalability:** Designing systems that can handle growing user bases and data volumes. 8. **Team Coordination:** Managing large, often geographically distributed development teams. 9. **Technology Evolution:** Keeping up with rapidly changing technologies, frameworks, and platforms. 10. **User Expectations:** Meeting high expectations for intuitive interfaces and seamless experiences.

2. What is extreme programming? Explain how the principles underlying extreme programming lead to the accelerated development of software.

[1+4]

Answer: Extreme Programming (XP): XP is an agile software development methodology introduced by Kent Beck that emphasizes customer satisfaction, continuous feedback, and rapid delivery of working software. It takes best practices to "extreme" levels. **Principles and How They Accelerate Development:** 1. **Fine-scale Feedback:** • **Pair Programming:** Two developers work together — one codes, one reviews. This catches defects immediately, reducing debugging time later. • **Planning Game:** Customers and developers collaborate on planning, ensuring alignment and reducing rework. • **Customer Presence:** On-site customer provides immediate clarifications, eliminating delays from waiting for responses. 2. **Continuous Process:** • **Continuous Integration:** Code is integrated and tested multiple times daily. Problems are caught immediately rather than at the end. • **Refactoring:** Continuous code improvement prevents technical debt accumulation, keeping the codebase maintainable. • **Small Releases:** Frequent small releases provide early value and rapid feedback. 3. **Shared Understanding:** • **Simple Design:** The simplest solution that works reduces development time and complexity. • **System Metaphor:** Shared conceptual framework helps team communicate efficiently. • **Collective Code Ownership:** Anyone can modify any code, eliminating bottlenecks when specific developers are unavailable. 4. **Programmer Welfare:** • **Sustainable Pace:** 40-hour weeks prevent burnout, maintaining consistent productivity. • **Coding Standards:** Consistent style reduces time spent understanding others' code. By emphasizing communication, simplicity, feedback, and courage, XP eliminates waste and focuses effort on delivering value quickly.

3. Define quality assurance? Under what circumstances would you recommend the use of staged representation of the CMM?

[2+3]

Answer: Software Quality Assurance (SQA): SQA is a set of activities that define and assess the adequacy of software processes to provide evidence that establishes confidence that the software processes are appropriate for and produce software products of suitable quality for their intended purposes. It includes the entire software development process — monitoring and improving the process, making sure standards and procedures are followed, and ensuring problems are found and fixed. **CMM Staged Representation:** The Capability Maturity Model (CMM) defines five maturity levels: 1. **Initial:** Ad hoc, chaotic processes 2. **Managed:** Basic project management processes established 3. **Defined:** Standard processes across the organization 4. **Quantitatively Managed:** Detailed metrics for process control 5. **Optimizing:** Continuous process improvement **Circumstances for Staged Representation:** 1. **Process Improvement Initiative:** When an organization wants a clear roadmap for improving software development capabilities step by step. 2. **Government or Defense Contracts:** Many government contracts (especially in the US) require specific CMM levels (typically Level 3 or higher). 3. **Large Organizations:** When standardizing processes across multiple teams or departments that currently work differently. 4. **Quality-Critical Systems:** For safety-critical software (medical devices, aerospace, nuclear systems) where process maturity directly impacts risk. 5. **Vendor Assessment:** When evaluating outsourced development partners — CMM level provides a standardized benchmark. 6. **Benchmarking:** Comparing organizational capability against industry standards. The staged model is preferred when a sequential, milestone-based improvement approach is desired, as opposed to the continuous representation which allows focusing on specific process areas.

4. What is architectural design? Explain layered model of software architecture with example.

[1+4]

Answer: Architectural Design: Architectural design is the process of defining a structured solution that meets all technical and operational requirements while optimizing common quality attributes such as performance, security, and manageability. It identifies major system components, their responsibilities, interfaces, and interactions. The architecture serves as the blueprint for both the system and the project developing it. **Layered Architecture Model:** The layered architecture pattern organizes software into distinct layers, each with a specific responsibility. Lower layers provide services to upper layers, and each layer is independent of layers above it. **Typical Layers:** 1. **Presentation Layer (UI Layer):** • Handles user interface and user interaction • Displays information and captures user input • Example: HTML/CSS/JavaScript in web apps, Android Activities 2. **Business Logic Layer (Application/Service Layer):** • Contains core business rules and logic • Processes data, makes decisions, coordinates workflows • Example: Order processing, payment validation, user authentication 3. **Data Access Layer (Persistence Layer):** • Handles database interactions • CRUD operations, query execution, ORM mapping • Example: Repository classes, DAO (Data Access Objects) 4. **Database Layer:** • Physical data storage • Example: MySQL, PostgreSQL, MongoDB **Example — E-Commerce Application: LayerComponents** PresentationProduct catalog page, shopping cart UI, checkout forms Business LogicCartService (calculates totals), OrderService (validates orders), PaymentService Data AccessProductRepository, OrderRepository, UserRepository DatabaseProducts table, Orders table, Users table **Advantages:** Separation of concerns, easier maintenance, testability, reusability, parallel development. **Disadvantages:** Performance overhead from layer transitions, potential for tight coupling if not designed carefully.

■ *Diagram Required:* A layered architecture diagram showing 4 horizontal layers: Presentation at top, then Business Logic, then Data Access, then Database at bottom, with downward arrows indicating dependency.

5. Differentiate between verification and validation. Explain why software inspection is an effective technique for discovering errors in software?

[3+2]

Answer: Verification vs Validation: AspectVerificationValidation DefinitionAre we building the product right?Are we building the right product? FocusProcess, documents, code conformity to specificationsFinal product meets user needs and expectations WhenThroughout development (reviews, inspections, walkthroughs)At the end or with user involvement (testing, UAT) ActivitiesStatic analysis, code reviews, requirement reviewsFunctional testing, beta testing, user acceptance ObjectiveEnsure software conforms to specificationsEnsure software meets intended use TechniquesWalkthroughs, inspections, auditsBlack-box testing, white-box testing, usability testing **Why Software Inspection is Effective:** 1. **Early Defect Detection:** Inspections find errors before testing, when they are cheaper to fix (up to 100x cheaper than post-deployment fixes). 2. **High Defect Detection Rate:** Studies show inspections can find 60-90% of defects, compared to 30-50% for testing alone. 3. **Diverse Perspectives:** Multiple reviewers bring different viewpoints — developers, testers, domain experts all contribute. 4. **Process Improvement:** Patterns in defects found during inspections reveal systematic problems in development processes. 5. **No Execution Required:** Inspections can be done on requirements, designs, and code without working software, enabling earlier feedback. 6. **Knowledge Sharing:** Team members learn from each other during inspections, improving overall team capability. 7. **Cost Effective:** Finding and fixing a defect during inspection costs a fraction of finding it during testing or in production.

6. What is software reengineering? List its advantages. Explain source code translation and reverse engineering approach in brief.

[1+1+3]

Answer: Software Reengineering: Software reengineering is the examination and alteration of a subject system to reconstitute it in a new form and the subsequent implementation of the new form. It involves analyzing, modifying, and reconstructing existing software systems to improve their maintainability, performance, or adaptability to new environments. It is not simply rewriting — it is a systematic transformation that preserves the system's functionality. **Advantages:** • Reduced risk compared to complete redevelopment • Lower cost than building from scratch • Preserves business knowledge embedded in the system • Improved maintainability and documentation • Better performance and reliability • Easier integration with modern systems • Extended system lifetime **Source Code Translation:** This is the automatic conversion of program source code from one programming language to another (e.g., COBOL to Java) or from one platform to another. Tools analyze the source code, parse it into an intermediate representation, and then generate equivalent code in the target language. Challenges include handling language-specific features, preserving semantics, and dealing with platform dependencies. **Reverse Engineering:** Reverse engineering is the process of analyzing a finished product (source code or executable) to understand its design, architecture, and functionality. It extracts design information from code when documentation is missing or outdated. Activities include: • Identifying system components and relationships • Recovering data models and control flow • Creating documentation from undocumented code • Understanding algorithms and business rules Reverse engineering is often the first step before forward engineering or reengineering, providing the understanding needed to make informed modifications.

7. What do you mean by software quality management? Explain software quality management activities in brief.

[2+3]

Answer: Software Quality Management (SQM): SQM is the collection of all processes and activities that ensure software meets its intended quality standards. It encompasses quality planning, quality assurance, quality control, and continuous improvement throughout the software lifecycle. SQM aims to deliver software that satisfies customer requirements, is fit for purpose, and is free from critical defects. **Software Quality Management Activities:** 1. **Quality Planning:** • Define quality goals and standards • Identify quality metrics and acceptance criteria • Determine quality assurance activities and resources • Create quality management plan 2. **Quality Assurance (QA):** • Process-focused activities to ensure development follows standards • Audits and reviews of processes and documentation • Training and process improvement initiatives • Establishing and maintaining quality standards 3. **Quality Control (QC):** • Product-focused testing and inspection activities • Unit testing, integration testing, system testing, acceptance testing • Defect tracking and reporting • Verification that product meets specifications 4. **Quality Improvement:** • Analyze defect data to identify root causes • Implement process changes to prevent recurring issues • Continuous monitoring and measurement • Apply frameworks like Six Sigma, CMMI, or TQM 5. **Configuration Management:** • Version control and change management • Ensuring traceability of requirements to code to tests • Managing releases and baselines 6. **Metrics and Measurement:** • Collect data on defects, effort, schedule, size • Calculate quality indicators (defect density, test coverage, MTBF) • Use metrics to drive decisions and improvements

Group C

Attempt any TWO questions. [2×10=20]

8. What is software process model? List different software process model. Explain how both the waterfall model of the software process and prototyping model can be accommodated in the spiral process model.

[2+2+6]

Answer: Software Process Model: A software process model is an abstract representation of a software process. It provides a structured approach to software development by defining the phases, activities, deliverables, and control mechanisms for building software systems. It serves as a roadmap guiding the project from conception to delivery and maintenance. **Different Software Process Models:** 1. Waterfall Model 2. V-Model 3. Incremental Model 4. Spiral Model 5. Prototyping Model 6. RAD (Rapid Application Development) 7. Agile/Scrum/XP 8. DevOps **Accommodating Waterfall and Prototyping in Spiral Model:** The **Spiral Model**, proposed by Barry Boehm, combines iterative development with systematic aspects of the waterfall model. Each loop of the spiral represents a phase of the software process and is divided into four quadrants: 1. **Determine Objectives, Alternatives, Constraints** 2. **Evaluate Alternatives, Identify and Resolve Risks** 3. **Develop and Verify Next-Level Product** 4. **Plan Next Phase Waterfall in Spiral:** Within each spiral loop (especially early loops with low risk), the development can follow a waterfall-like sequence. For well-understood requirements with clear objectives, the third quadrant can execute requirements → design → implementation → testing in a linear fashion. The waterfall's discipline of completing one phase before moving to the next fits within a single spiral iteration when risks are minimal and requirements are stable. **Prototyping in Spiral:** When risks are high and requirements are unclear (typically in early spirals), the third quadrant employs prototyping. A quick prototype is built to explore alternatives, validate requirements with users, and reduce uncertainty. Based on user feedback, the prototype evolves or is discarded. This directly maps to the risk-resolution focus of the second quadrant — prototyping is the primary technique for resolving requirement risks. The spiral model's flexibility allows it to incorporate different approaches in different iterations based on risk assessment — waterfall for low-risk phases, prototyping for high-risk uncertain areas.

■ *Diagram Required:* Spiral model diagram showing 4 quadrants: (1) Objectives, (2) Risk Analysis, (3) Development & Test, (4) Planning. Multiple loops expanding outward with increasing completeness.

9. What do you mean by software requirement? Differentiate between functional requirement and non-functional requirement. List the functional requirements of online examination system and represent it in use case diagram.

[2+4+4]

Answer: Software Requirement: A software requirement is a condition or capability needed by a user to solve a problem or achieve an objective. It can be a condition or capability that must be met or possessed by a system or system component to satisfy a contract, standard, specification, or other formally imposed documents. Requirements form the foundation for software design, implementation, testing, and acceptance.

Aspect	Functional Requirements	Non-Functional Requirements
Definition	What the system should do — specific behaviors, features, functions	How the system should behave — quality attributes, constraints
Focus	Business logic, data processing, user operations	Performance, security, usability, reliability
Examples	User login, data validation, report generation	Response time < 2s, 99.9% uptime, supports 1000 concurrent users
Testability	Verified by functional testing	Verified by performance testing, security audits
Source	Business rules, user stories, use cases	Quality standards, technical constraints, regulations
Documentation	Use cases, user stories, functional specs	Quality attribute scenarios, SLAs, architecture decisions

Functional Requirements of Online Examination System:

- User Management:** Registration, login, profile management for students, instructors, and administrators
- Exam Creation:** Instructors can create exams with different question types (MCQ, essay, coding)
- Question Bank:** Store, categorize, and manage questions by subject, difficulty, and topic
- Exam Scheduling:** Set exam dates, durations, availability windows, and attempt limits
- Exam Taking:** Students can take scheduled exams with timer, navigation, and auto-save
- Proctoring:** Browser lockdown, random photo capture, tab switching detection
- Auto-grading:** Automatic scoring for objective questions; rubric-based grading for subjective
- Result Publication:** Display scores, rankings, detailed analytics to students and instructors
- Report Generation:** Generate performance reports, statistics, and analytics dashboards
- Notification System:** Email/push notifications for exam schedules, results, and deadlines

Use Case Diagram Description:

- Actors:** Student, Instructor, Administrator, System
- Student use cases:** Register, Login, View Schedule, Take Exam, View Result, Download Certificate
- Instructor use cases:** Login, Create Exam, Manage Question Bank, Schedule Exam, Review Answers, Publish Results, Generate Reports
- Administrator use cases:** Manage Users, Configure System, Monitor Exams, Generate Analytics, Backup Data
- System use cases:** Send Notifications, Auto-grade, Proctor Exam, Generate Reports

■ *Diagram Required:* Use case diagram with three actors (Student, Instructor, Admin) on the left. System boundary labeled 'Online Examination System' contains use cases: Register, Login, Take Exam, View Result, Create Exam, Manage Questions, Schedule Exam, Review Answers, Publish Results, Auto-grade, Send Notifications, Generate Reports. Associations connect actors to relevant use cases.

10. a) What is risk management? Explain risk management process in software engineering with block diagram. b) List good programming practices. Explain CASE and its type with example.

[1+4+2+3]

Answer: a) Risk Management: Risk management is the process of identifying, analyzing, and responding to risk factors throughout the life of a project to minimize their negative impact on project objectives. In software engineering, risks can threaten schedule, budget, quality, or customer satisfaction. **Risk Management Process:** 1. **Risk Identification:** Discover potential risks through brainstorming, checklists, historical data, and expert judgment. Categories: project risks, technical risks, business risks. 2. **Risk Analysis (Assessment):** Evaluate each risk's probability of occurrence and potential impact. Prioritize risks based on severity (Probability × Impact). Create risk register. 3. **Risk Planning (Response):** Develop strategies for each significant risk: • **Avoidance:** Change plans to eliminate the risk • **Minimization:** Reduce probability or impact • **Contingency:** Plan alternative actions if risk occurs • **Acceptance:** Acknowledge and monitor 4. **Risk Monitoring:** Continuously track identified risks, detect new risks, and evaluate effectiveness of response strategies throughout the project. **Block Diagram Description:**

```
graph LR; A[Risk Identification] --> B[Risk Analysis]; B --> C[Risk Planning]; C --> D[Risk Monitoring]; D --> A; D --> E[Feedback Loop]; E --> A;
```

b) Good Programming Practices: 1. **Meaningful Naming:** Variables, functions, and classes should have descriptive names 2. **Comments and Documentation:** Explain why, not what — code should be self-documenting 3. **Modularity:** Break code into small, reusable functions with single responsibility 4. **Error Handling:** Anticipate and handle exceptions gracefully 5. **Code Reviews:** Peer review all code before integration 6. **Version Control:** Use Git or similar for tracking changes 7. **Testing:** Write unit tests alongside production code (TDD) 8. **Consistent Formatting:** Follow style guides (PEP 8, Google Style) 9. **DRY Principle:** Don't Repeat Yourself — avoid duplication 10. **KISS:** Keep It Simple, Stupid — simplest solution is often best **CASE (Computer-Aided Software Engineering):** CASE refers to software tools that automate activities in the software development lifecycle, improving productivity and quality. **Types of CASE Tools:** 1. **Upper CASE (Front-End):** Support early lifecycle phases • Requirements tools: IBM Rational RequisitePro • Design tools: Enterprise Architect, Visio, StarUML • Analysis tools: Data flow diagram editors 2. **Lower CASE (Back-End):** Support implementation and maintenance • Code generators • Compilers and debuggers • Testing tools: JUnit, Selenium • Configuration management: Git, Jenkins 3. **Integrated CASE (I-CASE):** Covers entire lifecycle • Examples: IBM Rational Suite, Microsoft Visual Studio **Example: StarUML** is an Upper CASE tool used for creating UML diagrams (class diagrams, use case diagrams, sequence diagrams) during the design phase. It helps visualize system architecture before coding begins, improving communication among team members and reducing design defects.

■ **Diagram Required:** Risk management block diagram: 4 rectangular boxes arranged horizontally: Risk Identification → Risk Analysis → Risk Planning → Risk Monitoring, with an arrow from Monitoring back to Identification forming a feedback loop.

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.